Advanced database management systems

Topic 1: Loose coupling based on triggers and jobs

István Vassányi, PhD

Pannon Egyetem
University of Pannonia

Overview

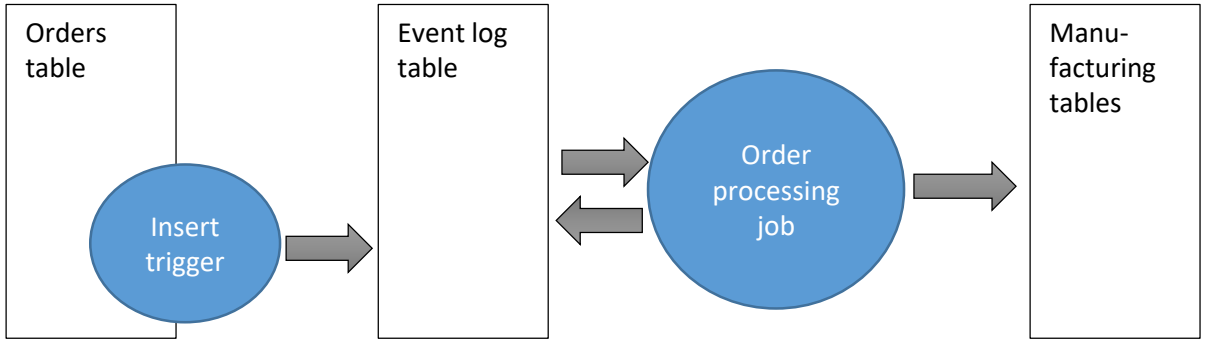
- We design and implement a database level loose coupling solution
- The idea is to separate the two connected systems as much as possible

After completing this topic:

- you will understand the loose vs tight coupling concept
- you will be able to design and work with common DML triggers
- you will be able to use cursors and stored procedures
- you will be able to design, start and manage database jobs

The case study

- A trading company with two divisions sharing the same database but running disparate information systems (IS)
- In the source system used by the trading division, business events are generated, like the placement of a new order or the modification of a previous order
- We want to propagate events into the IS of the shipping and logistics division
- The trading IS has no API
- We want an automated and robust mechanism
- The event propagation process should not at the least interfere with the source system
  - any eventual processing error should not be propagated back to the source system
- -> we choose a database level loose coupling solution

The architecture

- We log the INSERT and UPDATE events on orders via a trigger in a log table
- We process these events in a stored procedure that reads the log
- The procedure uses a cursor to iterate the new events since the last processing
- The procedure is called by a database job
- The job is scheduled to run off-peak hours